MEMORYMIX

Introduction

MemoryMix API

- Auth
  - GET GET Random Songs
  - GET GET Hardcoded Songs
  - POST POST Test Login — Hardcoded
  - POST POST Test Login — Random
- Memories
  - POST POST Create Memory
  - GET GET All Memories
  - GET GET Single Memory
  - DEL DELETE Memory
- Mappings
  - PUT PUT Swap Matched Track
- Songs
  - GET GET Song History

MemoryMix

MemoryMix API

MemoryMix is an API for creating and managing personal memories, each enriched with music through mood-based track matching.

Please make sure to view full API Document that is visible in the Github ReadME to see all responses and requests for each endpoint.

Getting Started

All requests require authentication. Use one of the **Auth** endpoints to obtain an access token, which will be stored as the `{access_token}` collection variable automatically.

Base URL

Plain Text

```
http://localhost:8000
```

Authentication

Use a Bearer token in the `Authorization` header:

Plain Text

```
Authorization: Bearer •••••••
```

Run one of the login requests first to populate `{access_token}` before calling any other endpoint.

Folders

Auth

Endpoints for obtaining a test access token. Two login variants are available:

- **POST Test Login — Hardcoded** – logs in with a fixed test user
- **POST Test Login — Random** – logs in with a randomly generated test user

Memories

CRUD operations for memory objects. Each memory includes a title, description, date, and an optional photo (base64-encoded).

Request	Method	Path
POST Create Memory	POST	`/memories`
GET All Memories	GET	`/memories`
GET Single Memory	GET	`/memories/{memory_id}`
DELETE Memory	DELETE	`/memories/{memory_id}`

> **Note:** When creating a memory, `memory_date` must be set to today's date, and `base_url` must be a real base64-encoded image string.

Mappings

Endpoints for managing the music track matched to a memory.

- **PUT Swap Matched Track** – replaces the matched track for a memory mapping. Use `mapping.id` from the GET Single Memory response as `mapping_id`, and a track `id` from `mood_candidates` in that same response.

Variables

Variable	Description
`access_token`	JWT token obtained after login. Set automatically by the Auth test scripts.
`memory_id`	ID of a memory. Set this after creating or retrieving a memory.
`mapping_id`	ID of a memory-track mapping. Found in the GET Single Memory response.

Response Codes

Response Code	Description
200 OK	The request was successful and the server returned the requested data. This status is typically returned for successful GET, PUT, or DELETE operations.
201 Created	The request was successful and a new resource was created. This status is typically returned when a POST request successfully creates a new item in the database.
401 Unauthorized	The request could not be processed because valid authentication credentials were not provided. This occurs when a user attempts to access a protected endpoint without a valid JWT token.
422 Unprocessable Entity	The request was received but the server could not process it due to invalid input data. In FastAPI this usually occurs when the request body, parameters, or data format do not match the expected schema defined by Pydantic models.

Auth

This route allows testers to access a pre-seeded list of songs, simulating a "music history" since they cannot connect to a real streaming service. Testers can choose either a set of well-known hardcoded songs or a random seed of tracks to demonstrate that the system can match any song to its description, not just a curated subset. After this, users are provided a real session for full API access, similar to a standard login flow.

GET GET Random Songs

http://localhost:8000/auth/test/preview-songs?mode=random

This GET request retrieves the history of pre-seeded songs. It allows you to view the previously fetched random songs, which serve as your working history for playback or analysis.

PARAMS

mode	random

Example Request — 200 OK - Successful Resp...

curl

```
curl --location 'http://localhost:8000/auth/test/preview-songs?mode=random'
```

Example Response — Body | Headers (4) — 200 OK

```
[
  {
    "rowid": 229693,
    "track_name": "Two Fingers",
    "artist_name": "Jethro Tull",
    "genre": "progressive rock",
    "spotify_id": "0iEz8W79xEGnn5i7fHan5a"
  },
```

View More

GET GET Hardcoded Songs

http://localhost:8000/auth/test/preview-songs

This GET request retrieves a list of hardcoded songs chosen by the system. These are well-known tracks that users can view before logging in, making it easier to verify if a description matches a song correctly.

Example Request — 200 OK - Successful Resp...

curl

```
curl --location 'http://localhost:8000/auth/test/preview-songs'
```

Example Response — Body | Headers (4) — 200 OK

```
[
  {
    "rowid": 231752,
    "track_name": "Calabasas",
    "artist_name": "Hey Marseilles",
    "genre": "rock",
    "spotify_id": "0l8jJat1Yn7dwQjdVL1ui4"
  },
```

View More

POST POST Test Login — Hardcoded

http://localhost:8000/auth/test/login

Creates a test authentication session using a predefined user and seeds the listening history with a fixed list of songs.

Authentication:
None required.

Returns:
JWT access token for subsequent API requests.

HEADERS

Content-Type	application/json

Body raw (json)

json

```json
{
  "mode": "hardcoded"
}
```

Example Request — 200 OK - Successful Resp...

curl

```
curl --location 'http://localhost:8000/auth/test/login' \
--header 'Content-Type: application/json' \
--data '{
  "mode": "hardcoded"
}'
```

Example Response — Body | Headers (4) — 200 OK

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIyIiwiZXhwIjoxNzczODQ0MDY3fQ.VQXQnq8b7aTk9
  "token_type": "bearer",
  "expires_in": 604800,
  "user": {
    "email": null,
    "display_name": "Tester",
    "id": 2,
```

View More

POST POST Test Login — Random

http://localhost:8000/auth/test/login

Creates a test authentication session using a predefined user and seeds the listening history with a random list of songs taken from GET Random Songs.

Authentication:
None required.

Returns:
JWT access token for subsequent API requests.

HEADERS

Content-Type	application/json

Body raw (json)

json

```json
{
    "mode": "random",
    "rowids": ["229693", "243624", "197053"]
}
```

Example Request — 200 OK - With Seeded So...

curl

```
curl --location 'http://localhost:8000/auth/test/login' \
--header 'Content-Type: application/json' \
--data '{
  "mode": "random",
  "rowids": [229693,243624,197053,220145,234080,200210,246054,189397,221303,209275,
  186256,193818,210710,187597,225704
  ]
}'
```

Example Response — Body | Headers (4) — 200 OK

json

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIyIiwiZXhwIjoxNzczODQ2OTI4fQ.nZNKNWP8RZceN
  "token_type": "bearer",
  "expires_in": 604800,
  "user": {
    "email": null,
    "display_name": "Tester",
    "id": 2,
```

View More

Memories

Memories allow users to capture a moment by posting a photo along with a description and the time it was taken. Each memory is then automatically matched with a song from the user's listening history on that day.

For example, if a user listened to *Mr. Blue Sky* on March 11th, 2026, and added the description "A beautiful day in the sun," the memory would be linked to that song due to a similarity in song mood and the description tone.

Users can submit new memories, retrieve a list of all past memories, view individual memories in detail, and delete memories if desired.

POST POST Create Memory

http://localhost:8000/memories

This endpoint allows users to create a new memory by submitting a **title**, **description**, **photo**, and the **time the photo was taken**. Once submitted, the memory is saved, and the description is converted into an embedding vector. The system then compares this embedding against the user's listening history for that day and identifies the song(s) with the closest match. Each candidate song is scored by similarity confidence, and the top match is automatically linked to the memory.

HEADERS

Authorization	Bearer •••••••
Content-Type	application/json

Body raw (json)

json

```
{
  "title": "Sunset at the Beach",
  "description": "Golden hour on a quiet beach, warm light reflecting off the water, waves gently rolling
  "memory_date": "2026-03-11T18:00:00",
  "photo": {
    "google_photo_id": "local-upload",
    "base_url": "",
    "filename": "test-photo.jpg",
```

View More

Example Request — 201 Created - Success Ex...

curl

```
curl --location 'http://localhost:8000/memories' \
--header 'Authorization: Bearer •••••••' \
--header 'Content-Type: application/json' \
--data '{
  "title": "Sunset at the Beach",
  "description": "Golden hour on a quiet beach, warm light reflecting off the water, waves gently rolling in
  "memory_date": "2026-03-11T18:00:00",
  "photo": {
```

View More

Example Response — Body | Headers (4) — 201 CREATED

json

```
{
  "title": "Sunset at the Beach",
  "description": "Golden hour on a quiet beach, warm light reflecting off the water, waves gently rolling in
  "memory_date": "2026-03-11T18:00:00",
  "id": 6,
  "user_id": 1,
  "created_at": "2026-03-12T22:31:10.482022",
  "updated_at": "2026-03-12T22:31:10.482028",
```

View More

GET GET All Memories

http://localhost:8000/memories

Retrieves all memories posted by the user, showing all photos matched to a recommended song.

HEADERS

Authorization	Bearer •••••••

Example Request — 200 OK - Success Example

curl

```
curl --location 'http://localhost:8000/memories' \
--header 'Authorization: Bearer •••••••'
```

Example Response — Body | Headers (4) — 200 OK

json

```
[
  {
    "title": "Summer Vacation",
    "description": "Was a lovely day in the sun. ",
    "memory_date": "2026-03-12T14:27:00",
    "id": 1,
    "user_id": 1,
    "created_at": "2026-03-12T14:27:50.945683",
```

View More

GET GET Single Memory

http://localhost:8000/memories/

Retrieves a single memory by ID. The memory in the memory/{memoryId} should be replaced with the `id` returned from the POST Create Memory response.

HEADERS

Authorization	Bearer •••••••

Example Request — 200 OK – Get Memory by ID

curl

```
curl --location 'http://localhost:8000/memories/2' \
--header 'Authorization: Bearer •••••••'
```

Example Response — Body | Headers (4) — 200 OK

json

```
{
  "title": "Sunset at the Beach",
  "description": "Golden hour on a quiet beach, warm light reflecting off the water, waves gently rolling in
  "memory_date": "2026-03-11T18:00:00",
  "id": 2,
  "user_id": 2,
  "created_at": "2026-03-20T14:19:36.771824",
  "updated_at": "2026-03-20T14:19:36.771829",
```

View More

DELETE DELETE Memory

http://localhost:8000/memories/

Deletes a memory by ID. The memory ID in /memories/{memoryid} should be replaced with the `id` returned from the POST Create Memory response.

HEADERS

Authorization	Bearer •••••••

Example Request — 200 OK - Delete Memory ...

curl

```
curl --location --request DELETE 'http://localhost:8000/memories/7' \
--header 'Authorization: Bearer •••••••'
```

Example Response — Body | Headers (4) — 200 OK

json

```json
{
  "message": "Memory deleted successfully"
}
```

Mappings

PUT PUT Swap Matched Track

http://localhost:8000/mappings/{mapping_id}

Swaps the matched track for a memory mapping. The `mapping_id` should use `mapping.id` from the GET Single Memory response. The track `id` value should come from any of the candidate songs in from that day. These are available from GET Song History response.

HEADERS

Authorization	Bearer •••••••
Content-Type	application/json

Body raw (json)

json

```json
{
  "track_id": 42
}
```

Example Request — 200 OK - Track Swap Suc...

curl

```
curl --location --request PUT 'http://localhost:8000/mappings/1' \
--header 'Authorization: Bearer •••••••' \
--header 'Content-Type: application/json' \
--data '{
  "track_id": 832
}'
```

Example Response — Body | Headers (4) — 200 OK

json

```
{
  "photo_id": 1,
  "track_id": 832,
  "is_auto_suggested": false,
  "confidence_score": 64,
  "id": 1,
  "memory_id": 1,
  "mood_text": "This song feels happy with lively, energetic movement. The atmosphere is joyful, uplifting,
```

View More

Songs

GET GET Song History

http://localhost:8000/lastfm/history

Retrieves the songs assigned during authentication that represent the user's listening history. These tracks are used to match memory descriptions to songs. The response also provides track numeric IDs, which are required when updating or swapping matched tracks via the **PUT Swap Matched Track** endpoint.

HEADERS

Authorization	Bearer •••••••

Example Request — 200 OK - Success Example

curl

```
curl --location 'http://localhost:8000/lastfm/history' \
--header 'Authorization: Bearer •••••••'
```

Example Response — Body | Headers (4) — 200 OK

json

```
[
  {
    "track_id": "1WUSs195It8jj78gYMD9CT",
    "track_name": "Hey You!!!",
    "artist_name": "The Cure",
    "album_name": "happy",
    "album_image_url": null,
    "played_at": "2026-03-12T12:30:00",
```

View More